

BABU BANARASI DAS

UNIVERSITY

Session:-2026-2027



SCHOOL OF COMPUTER APPLICATIONS

No SQL Project

Student Management System Using MongoDB

Submitted to:

Mr. Harendra Singh

Department: BCA (DS & AI)

Date: 25th September

Submitted by:

SHABBAR ABBAS ZAIDI

Roll no: (15)

University roll no:

1250258407

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher Mr. Harendra Singh for giving me the opportunity to work on this NoSQL project titled “Student Management System Using MongoDB”.

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to create, retrieve, update, Sorting and delete data. I also learned how different query operators such as greater than, less than, equal to, AND, and OR are used in MongoDB.

I am thankful to my teacher for the guidance and support provided throughout the completion of this project.

INTRODUCTION

MongoDB is a popular NoSQL database management system that stores data in the form of documents. Unlike traditional relational databases, MongoDB does not require data to be stored in fixed rows and columns.

In this project, a Student Management System has been created using MongoDB. The database contains information about 50 students, including their roll number, name, age, marks, and city.

Different MongoDB operations have been performed on the student data, such as inserting, displaying, updating, deleting, sorting, counting, and searching records. Comparison operators such as \$eq, \$gt, \$lt, \$gte, and \$lte have also been demonstrated.

OBJECTIVES

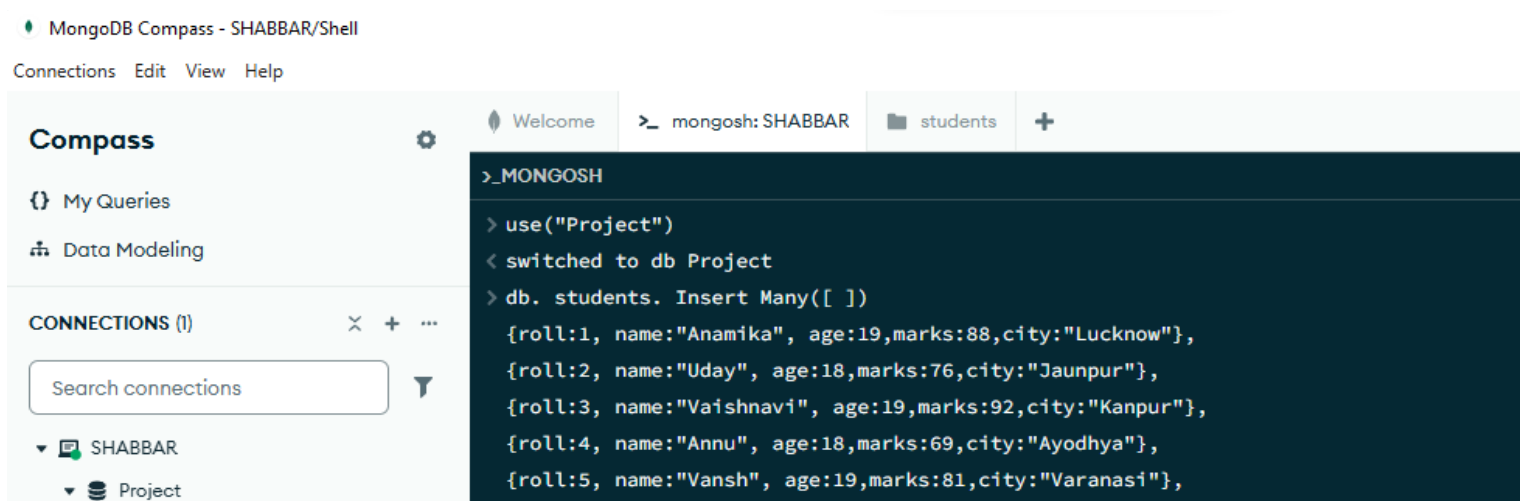
- **To understand the concept of NoSQL databases.**
- **To learn the basic features of MongoDB.**
- **To create a database and collection.**
- **To insert 50 student records.**
- **To perform CRUD operations.**
- **To use comparison operators.**
- **To perform sorting and filtering.**
- **To understand AND and OR operations.**

1. CREATE DATABASE

MongoDB Query:

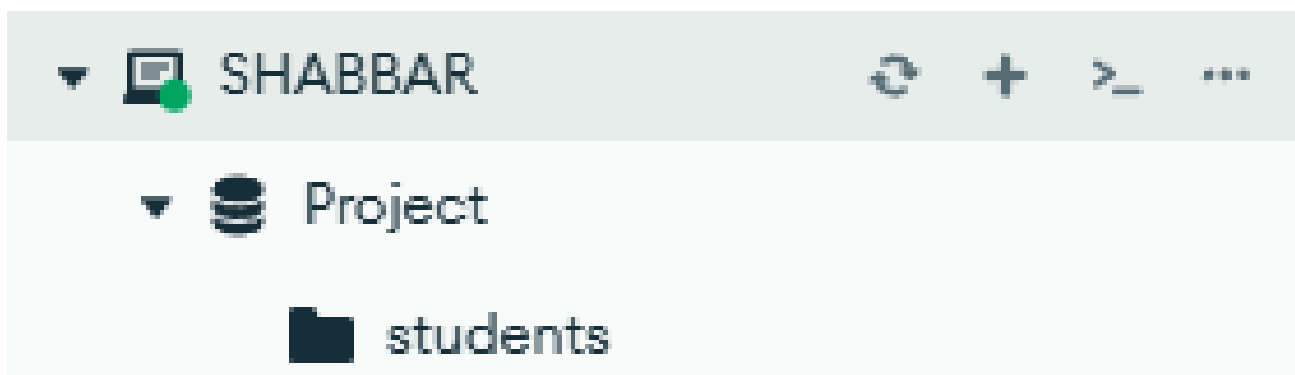
use Project

Explanation: Creates or switches to the Project database.



2. CREATE COLLECTION

Explanation: Creates a collection named *students*.



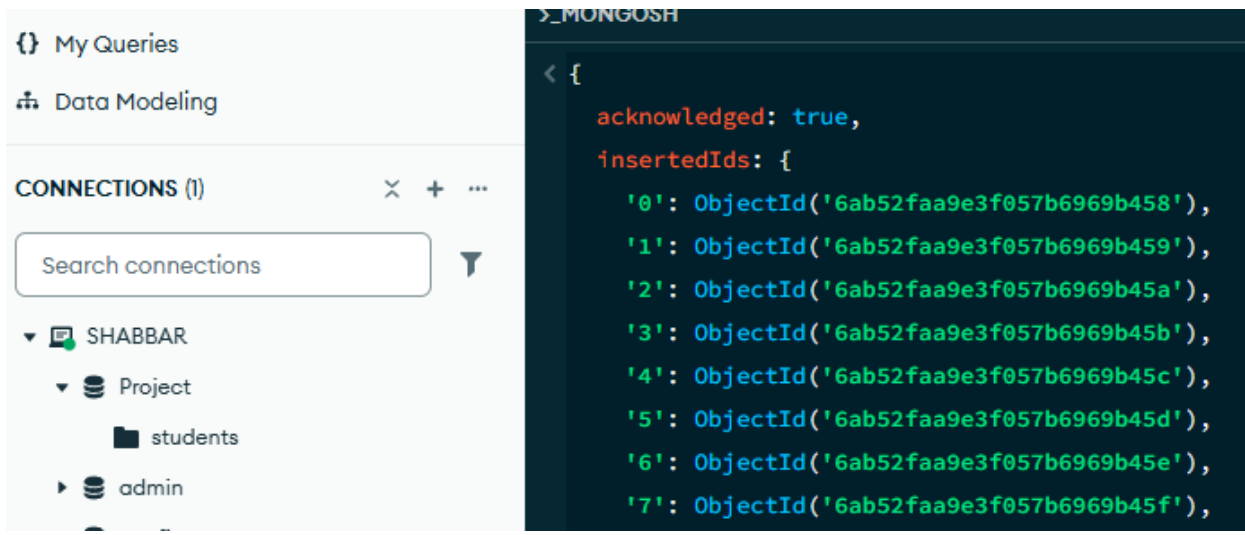
3.INSERT 50 STUDENTS

MongoDB Query:

```
db.students.insertMany([
  {roll:1, name:"Anamika", age:19,marks:88,city:"Lucknow"},
  {roll:2, name:"Uday", age:18,marks:76,city:"Jaunpur"},
  {roll:3, name:"Vaishnavi", age:19,marks:92,city:"Kanpur"},
  {roll:4, name:"Annu", age:18,marks:69,city:"Ayodhya"},
  {roll:5, name:"Vansh", age:19,marks:81,city:"Varanasi"},
  {roll:6, name:"Amrita", age:20,marks:85,city:"Lucknow"},
  {roll:7, name:"Naveen", age:19,marks:73,city:"Prayagraj"},
  {roll:8, name:"Pratik", age:19,marks:90,city:"Lucknow"},
  {roll:9, name:"Sangita", age:20,marks:78,city:"Gorakhpur"},
  {roll:10, name:"Vivek", age:18,marks:65,city:"Kanpur"},
  {roll:11, name:"Tarkeshwar", age:20,marks:71,city:"Jaunpur"},
  {roll:12, name:"Shreya", age:19,marks:95,city:"Lucknow"},
  {roll:13, name:"Akshita", age:18,marks:84,city:"Ayodhya"},
  {roll:14, name:"Shruti", age:19,marks:87,city:"Prayagraj"},
  {roll:15, name:"Kartik", age:20,marks:74,city:"Varanasi"},
  {roll:16, name:"Sejal", age:18,marks:89,city:"Lucknow"},
  {roll:17, name:"Astha", age:19,marks:82,city:"Kanpur"},
  {roll:18, name:"Jai", age:18,marks:58,city:"Gonda"},
  {roll:19, name:"Harshita", age:20,marks:91,city:"Lucknow"},
  {roll:20, name:"Omkar", age:19,marks:63,city:"Bareilly"},
  {roll:21, name:"Ayush", age:18,marks:77,city:"Lucknow"},
  {roll:22, name:"Saumya", age:19,marks:86,city:"Jaunpur"},
  {roll:23, name:"Vidya", age:20,marks:72,city:"Kanpur"},
  {roll:24, name:"Adarsh", age:19,marks:80,city:"Ayodhya"},
  {roll:25, name:"Anmol", age:18,marks:67,city:"Prayagraj"},
  {roll:26, name:"Anushka", age:20,marks:94,city:"Lucknow"},
```

```
{roll:27, name: "Ananya", age:19,marks:88,city:"Varanasi"},
{roll:28, name: "Bhaskar", age:18,marks:61,city:"Gorakhpur"},
{roll:29, name: "Rohan", age:19,marks:75,city:"Kanpur"},
{roll:30, name: "Nishant", age:20,marks:79,city:"Lucknow"},
{roll:31, name: "Vineet", age:18,marks:68,city:"Jaunpur"},
{roll:32, name: "Anika", age:19,marks:90,city:"Lucknow"},
{roll:33, name: "Jai", age:20,marks:55,city:"Ayodhya"},
{roll:34, name: "Lakshya", age:18,marks:83,city:"Kanpur"},
{roll:35, name: "Krishna", age:19,marks:96,city:"Prayagraj"},
{roll:36, name: "Rumana", age:20,marks:70,city:"Lucknow"},
{roll:37, name: "Priyanshu", age:18,marks:81,city:"Varanasi"},
{roll:38, name: "Aditya", age:19,marks:66,city:"Gorakhpur"},
{roll:39, name: "Dhruv", age:20,marks:93,city:"Lucknow"},
{roll:40, name: "Nivedita", age:19,marks:85,city:"Kanpur"},
{roll:41, name: "Meenu", age:18,marks:74,city:"Jaunpur"},
{roll:42, name: "Akanksha", age:20,marks:89,city:"Lucknow"},
{roll:43, name: "Sarika",age:19,marks:62,city:"Ayodhya"},
{roll:44, name: "Anchal",age:18,marks:78,city:"Prayagraj"},
{roll:45, name: "Umra", age:20,marks:84,city:"Lucknow"},
{roll:46, name: "Nashra", age:19,marks:73,city:"Kanpur"},
{roll:47, name: "Aman", age:18,marks:59,city:"Gonda"},
{roll:48, name: "Priya", age:20,marks:87,city:"Lucknow"},
{roll:49, name: "Rahul", age:19,marks:64,city:"Bareilly"},
{roll:50, name: "Neha", age:18,marks:91,city:"Varanasi"}
```

])

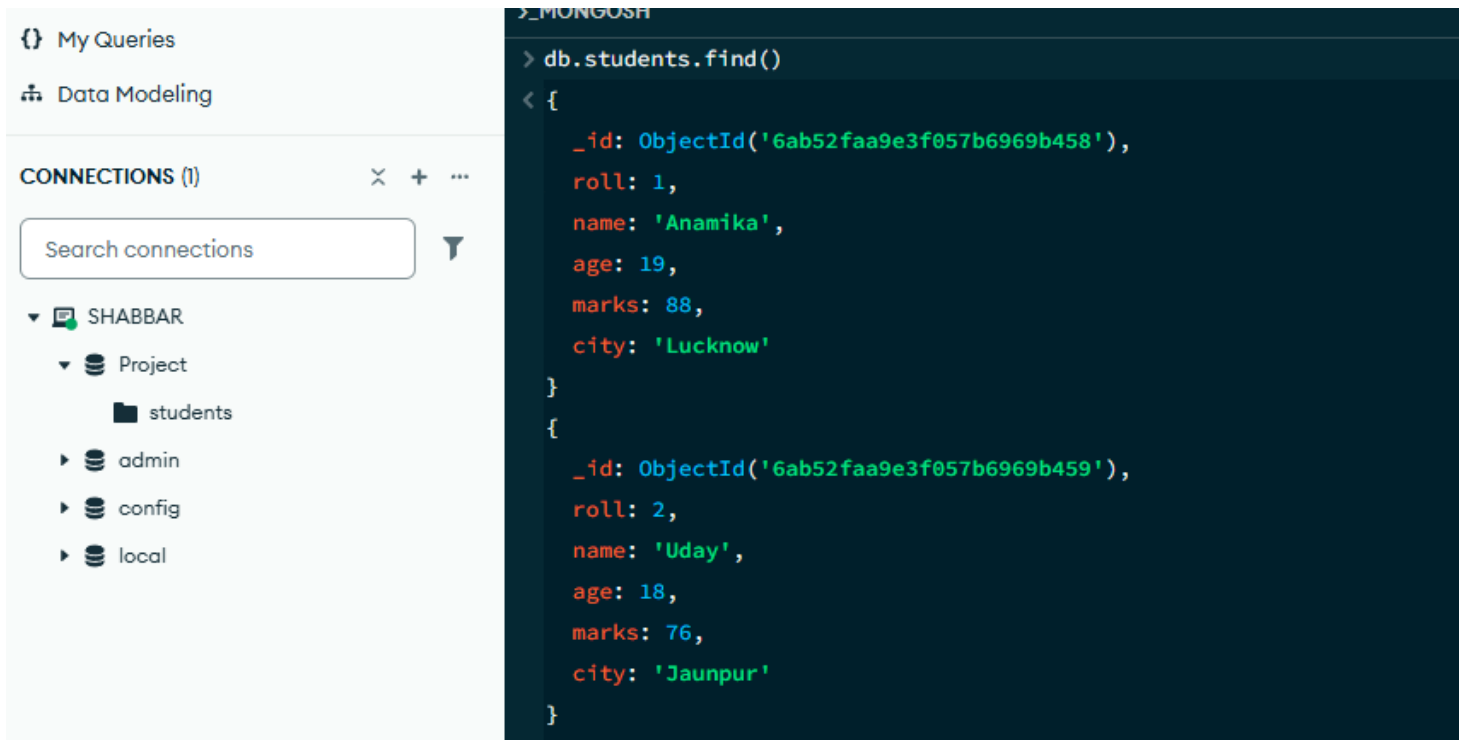


4. Display all students

Explanation: Retrieves all students data from students collection.

MongoDB Query:

db.students.find()

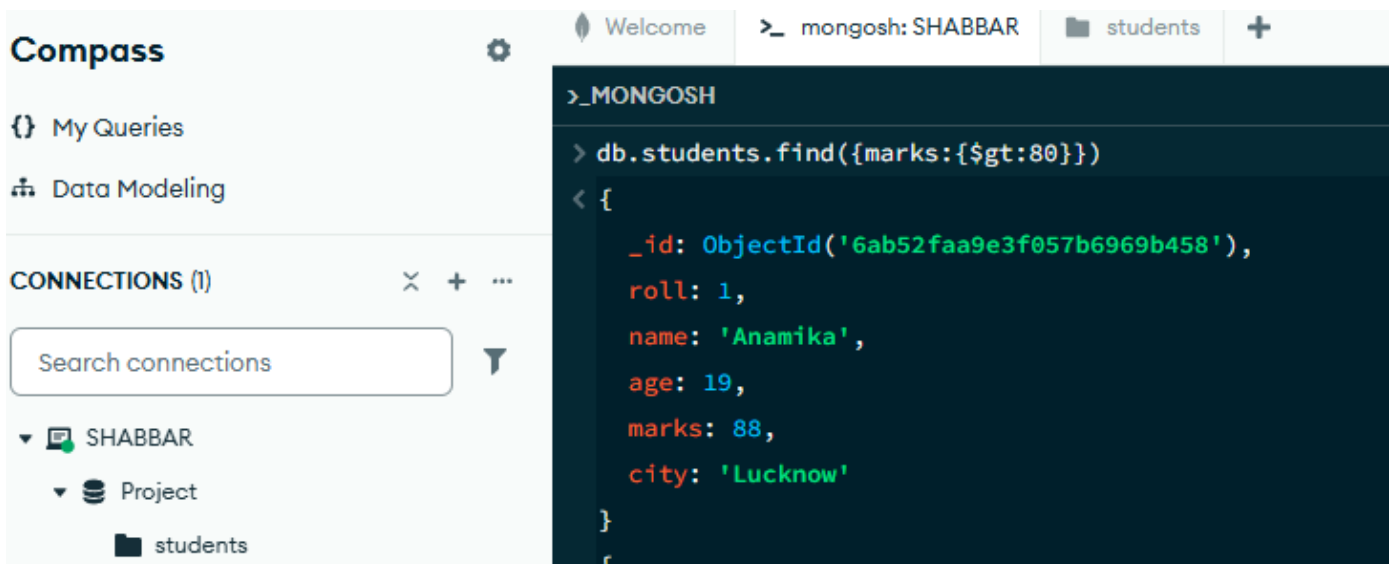


5. GREATER THAN - \$gt

Explanation: finds students whose marks are greater than a given number.

MongoDB Query:

`db.students.find({marks:{$gt:80}})`



6. LESS THAN - \$lt

Explanation : finds students whose marks are less than the given number.

MongoDB Query:

db.students.find({marks:{\$lt:80}})

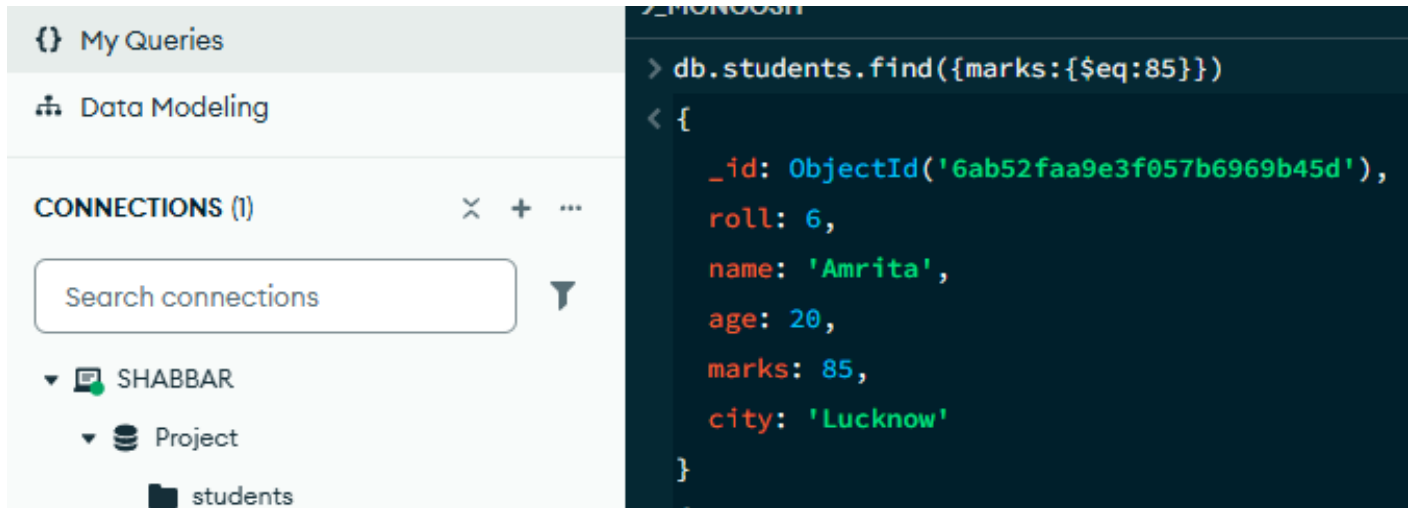
```
> db.students.find({marks:{$lt:80}})
< {
  _id: ObjectId('6ab52faa9e3f057b6969b459'),
  roll: 2,
  name: 'Uday',
  age: 18,
  marks: 76,
  city: 'Jaunpur'
}
```

7.EQUAL TO - \$eq

Explanation: finds students whose marks are equal to the given number.

MongoDB Query:

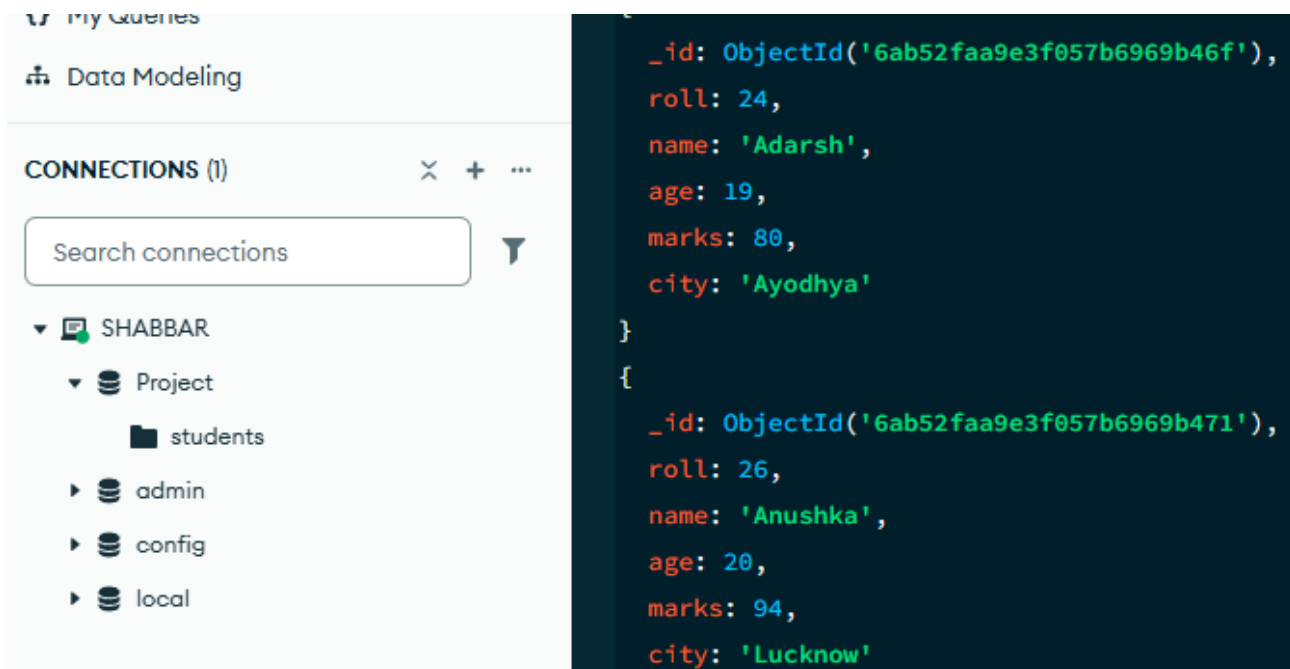
db.students.find({marks:{\$eq:85}})



8.GREATER THAN EQUAL TO - \$gte

Explanation: finds students whose marks are equal to or greater than the given number.

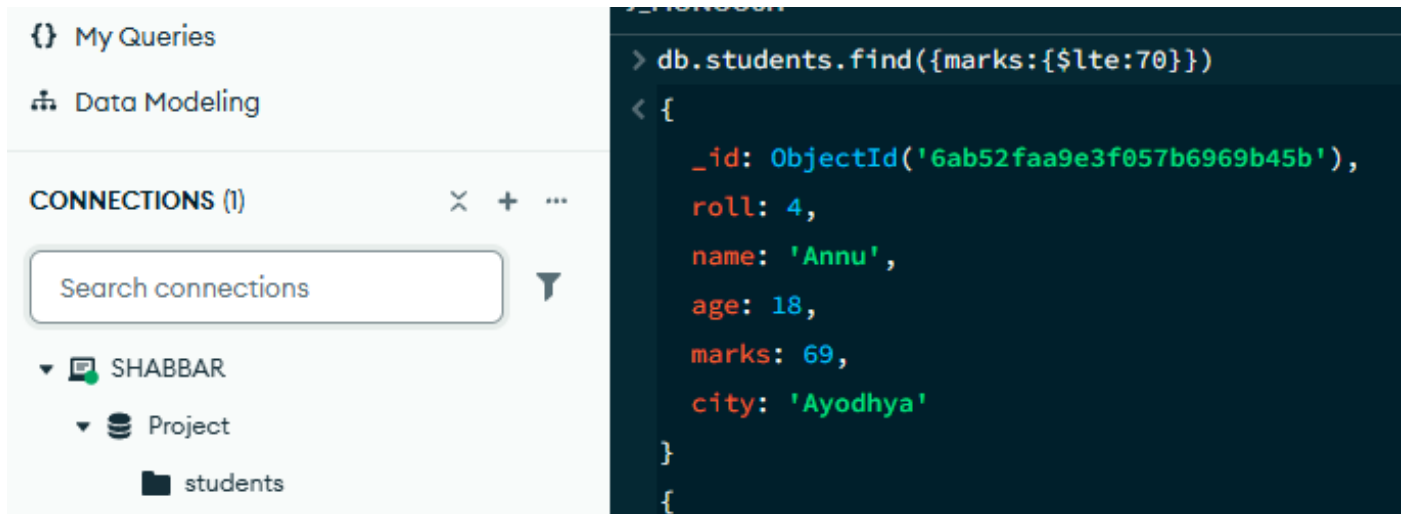
MongoDB Query: *db.students.find ({marks:{\$gte:80}})*



8. LESS THAN EQUAL TO - \$lte

Explanation: finds students whose marks are less than or equal to the given number.

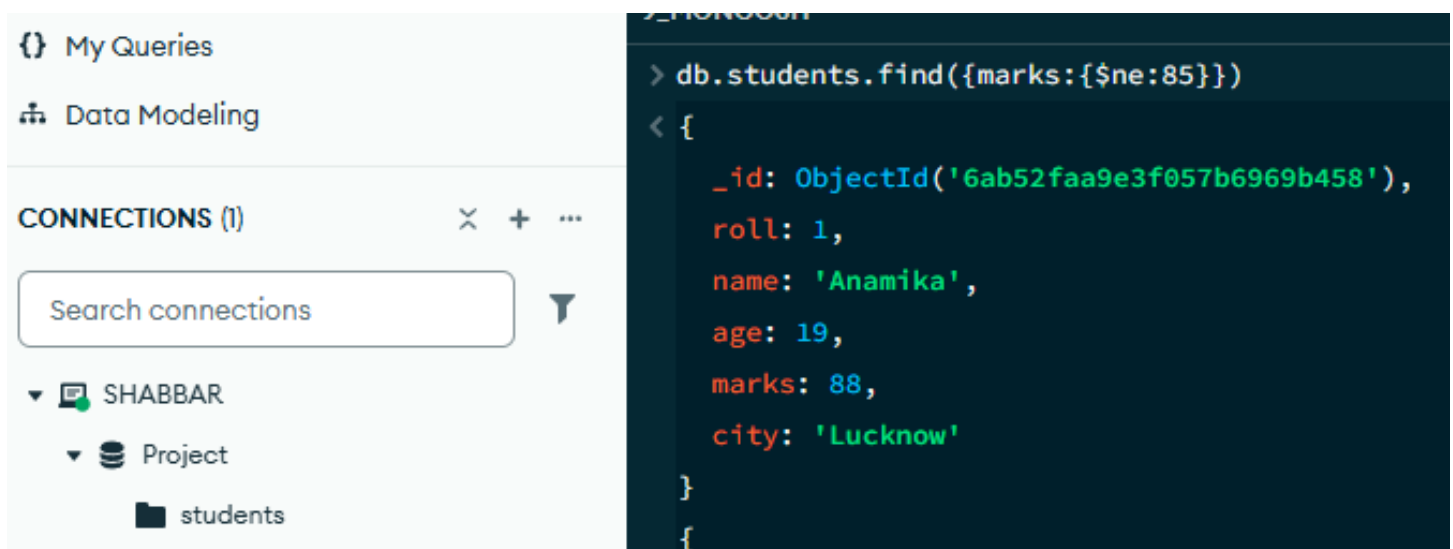
MongoDB Query: `db.students.find({marks:{$lte:70}})`



9. NOT EQUAL TO - \$ne

Explantion: finds students whose marks are not equal to the given number.

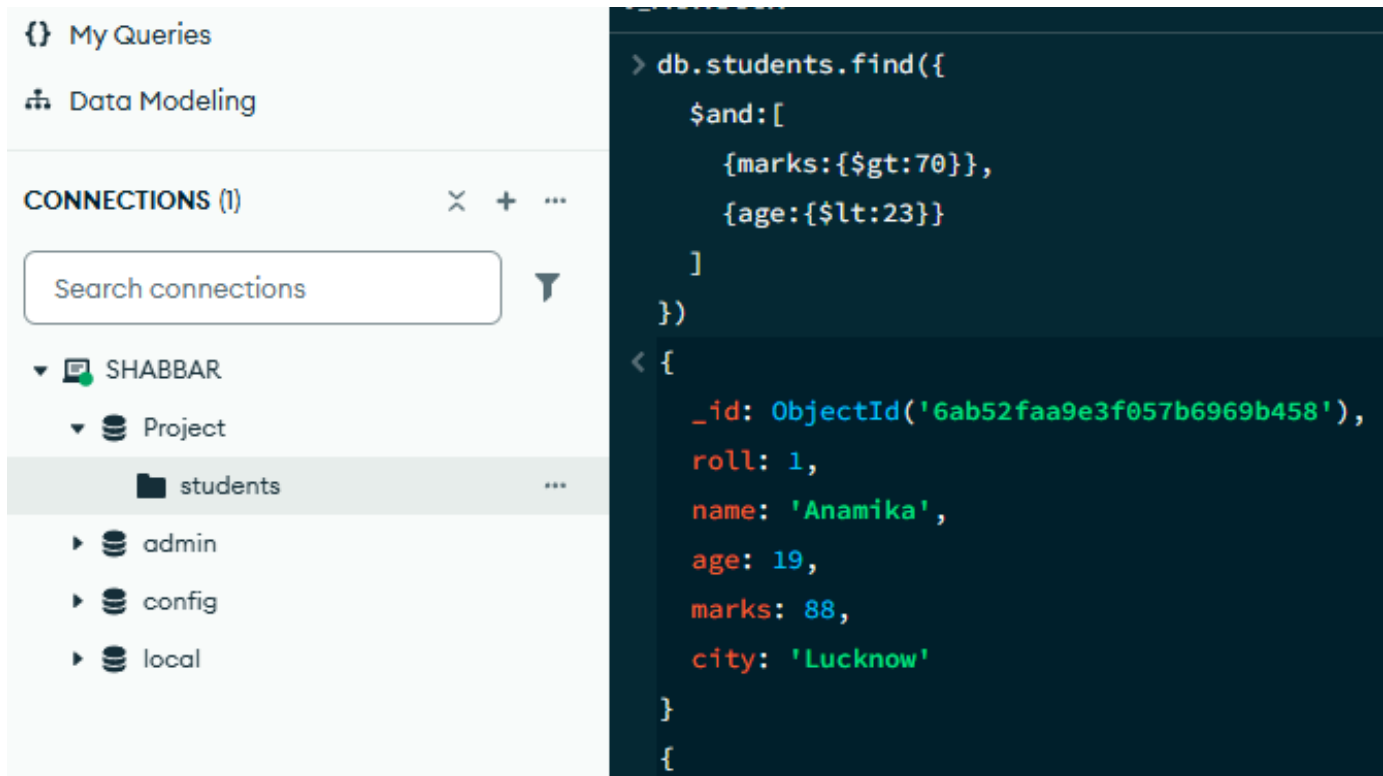
MongoDB Query: `db.students.find({marks:{$ne:85}})`



10. AND OPERATIONS - \$and

Explanation: Finds students data that satisfies both the given conditions.

MongoDB Query:



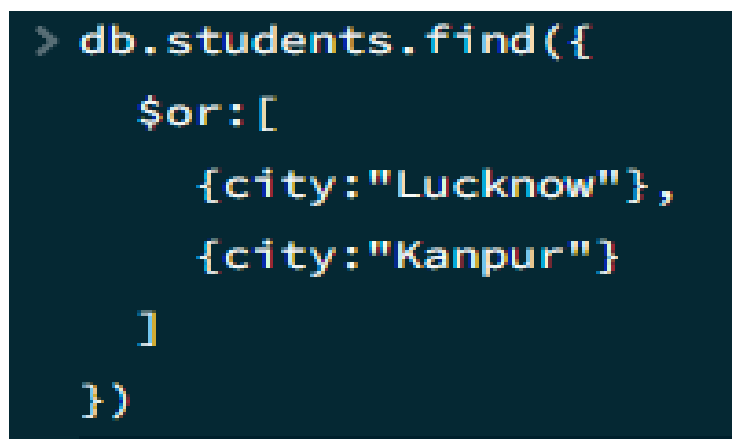
The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS (1)' panel shows a connection named 'SHABBAR' with a 'Project' folder containing a 'students' collection. The main panel displays a query in the 'Query' tab: `db.students.find({ $and: [{ marks: { $gt: 70 } }, { age: { $lt: 23 } } ] })`. The 'Results' tab shows a single document: `{ _id: ObjectId('6ab52faa9e3f057b6969b458'), roll: 1, name: 'Anamika', age: 19, marks: 88, city: 'Lucknow' }`.

```
> db.students.find({
  $and: [
    {marks:{$gt:70}},
    {age:{$lt:23}}
  ]
})
< {
  _id: ObjectId('6ab52faa9e3f057b6969b458'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
```

11. OR OPERATION - \$or

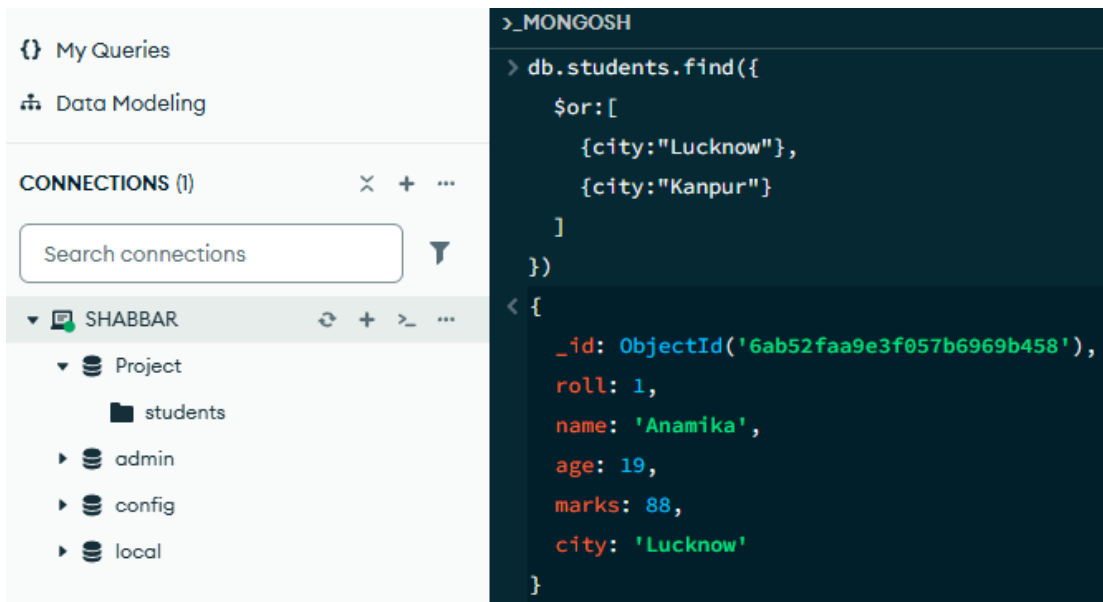
Explanation: Finds students data that satisfies one the given conditions.

MongoDB Query:



The screenshot shows a MongoDB query editor with the following query: `db.students.find({ $or: [{ city: "Lucknow" }, { city: "Kanpur" } ] })`.

```
> db.students.find({
  $or: [
    {city:"Lucknow"},
    {city:"Kanpur"}
  ]
})
```



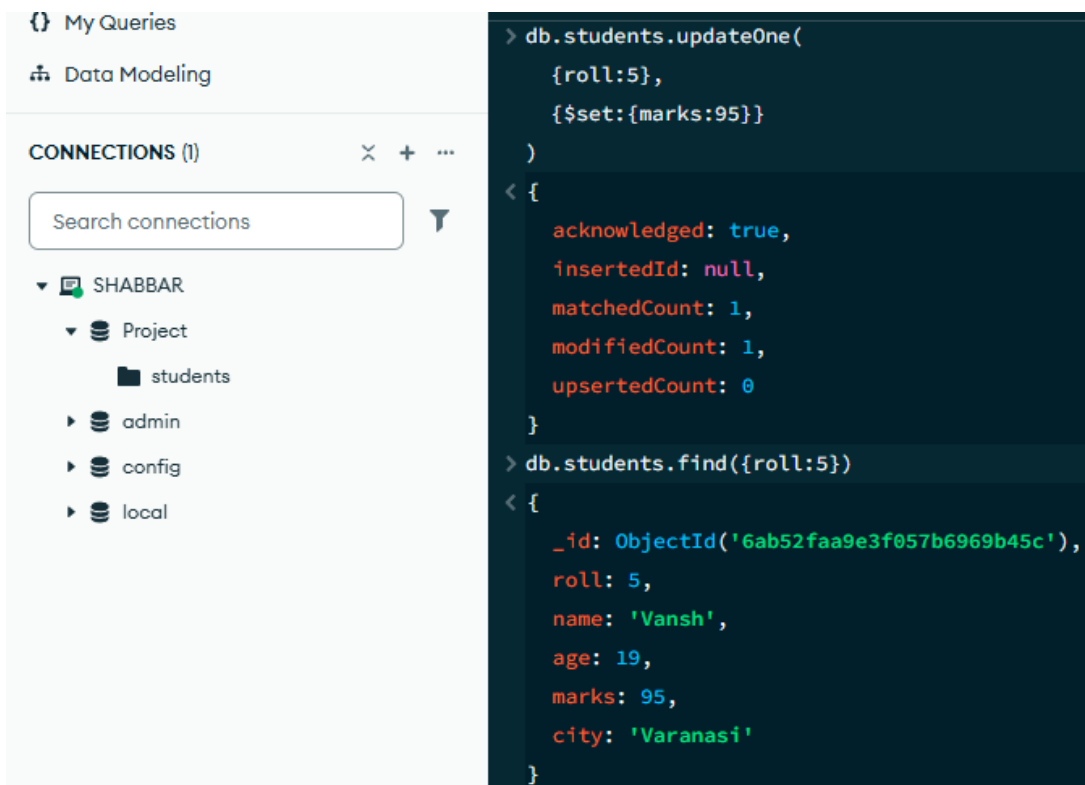
The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS (1)' panel shows a connection named 'SHABBAR' with a database 'Project' containing a collection 'students'. The main panel displays a query in the MongoDB Shell:

```
>_MONGOSH
> db.students.find({
  $or:[
    {city:"Lucknow"},
    {city:"Kanpur"}
  ]
})
< {
  _id: ObjectId('6ab52faa9e3f057b6969b458'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
```

12.UPDATE OPERATION

Explanation: Updates the older information to the new information of the given data.

MongoDB Query:



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS (1)' panel shows a connection named 'SHABBAR' with a database 'Project' containing a collection 'students'. The main panel displays a query in the MongoDB Shell:

```
> db.students.updateOne(
  {roll:5},
  {$set:{marks:95}}
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.students.find({roll:5})
< {
  _id: ObjectId('6ab52faa9e3f057b6969b45c'),
  roll: 5,
  name: 'Vansh',
  age: 19,
  marks: 95,
  city: 'Varanasi'
}
```

13. DELETE ONE OPERATION

Explanation: deletes one document that matches the given data.

MongoDB Query: *db.students.deleteOne({roll:10})*

```
> db.students.deleteOne({roll:10})
< {
  acknowledged: true,
  deletedCount: 1
}
> db.students.find({roll:10})
<
```

14. DELETE MANY OPERATION

Explanation: Deletes many documents that matches the given data.

MongoDB Query:

```
> db.students.deleteMany({
  marks: {$lt:60}
})
< {
  acknowledged: true,
  deletedCount: 3
}
```

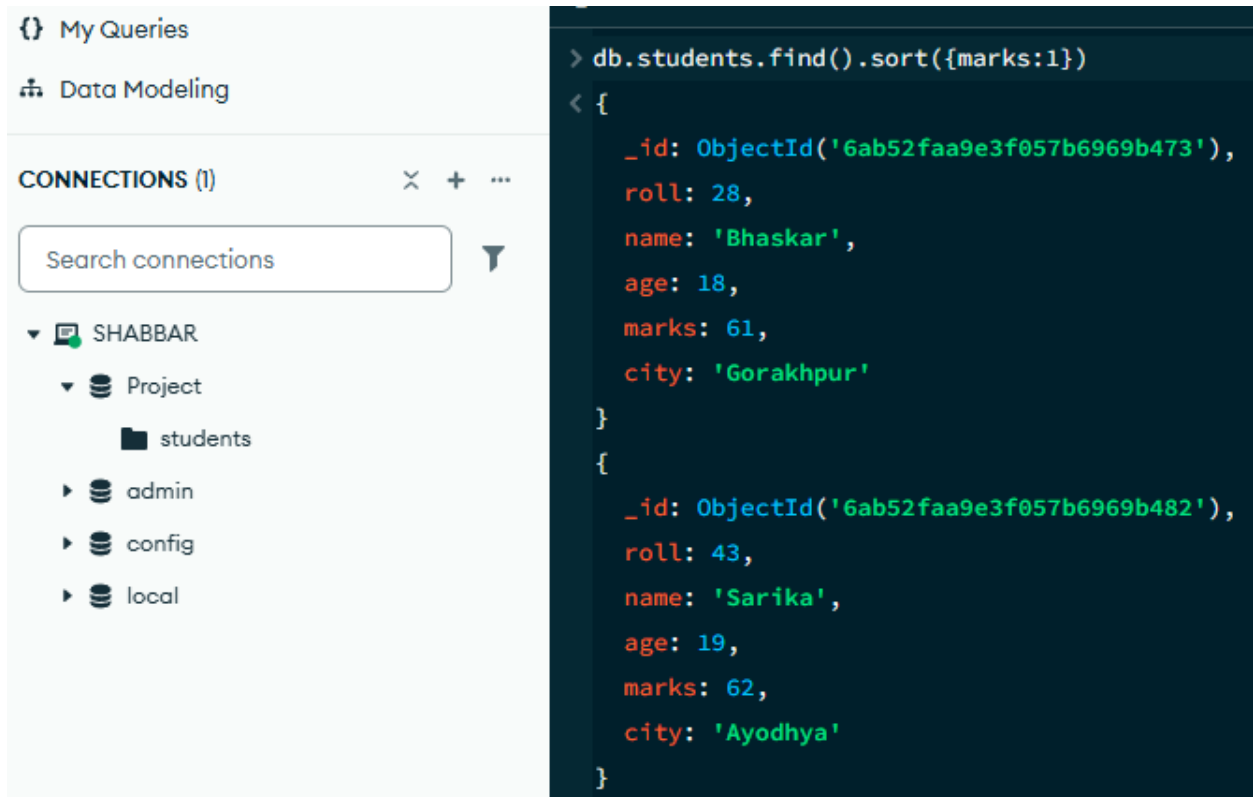
15. SORT OPERATION

Explanation: sorts students on the basis of their marks .

ASCENDING ORDER:-

Use 1 for ascending order.

MongoDB Query: *db.students.find().sort({marks:1})*

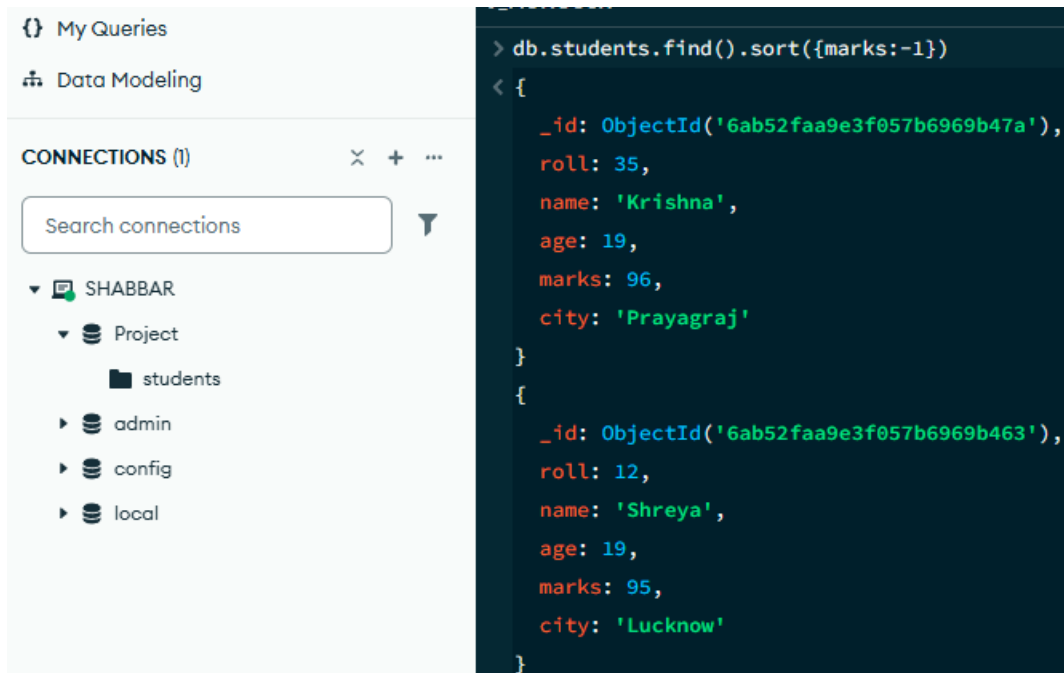


DESCENDING ORDER:-

Use -1 for descending order.

MongoDB Query:

db.students.find().Sort({marks:-1})



16. LIMIT OPERATION

Explanation: Displays only limited number of data. (Limit should be provided in the query).

MongoDB Query: `db.students.find().limit(5)`

```
> db.students.find().limit (5)
< {
  _id: ObjectId('6ab52faa9e3f057b6969b458'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6ab52faa9e3f057b6969b459'),
  roll: 2,
  name: 'Uday',
  age: 18,
  marks: 76,
  city: 'Jaunpur'
}
```

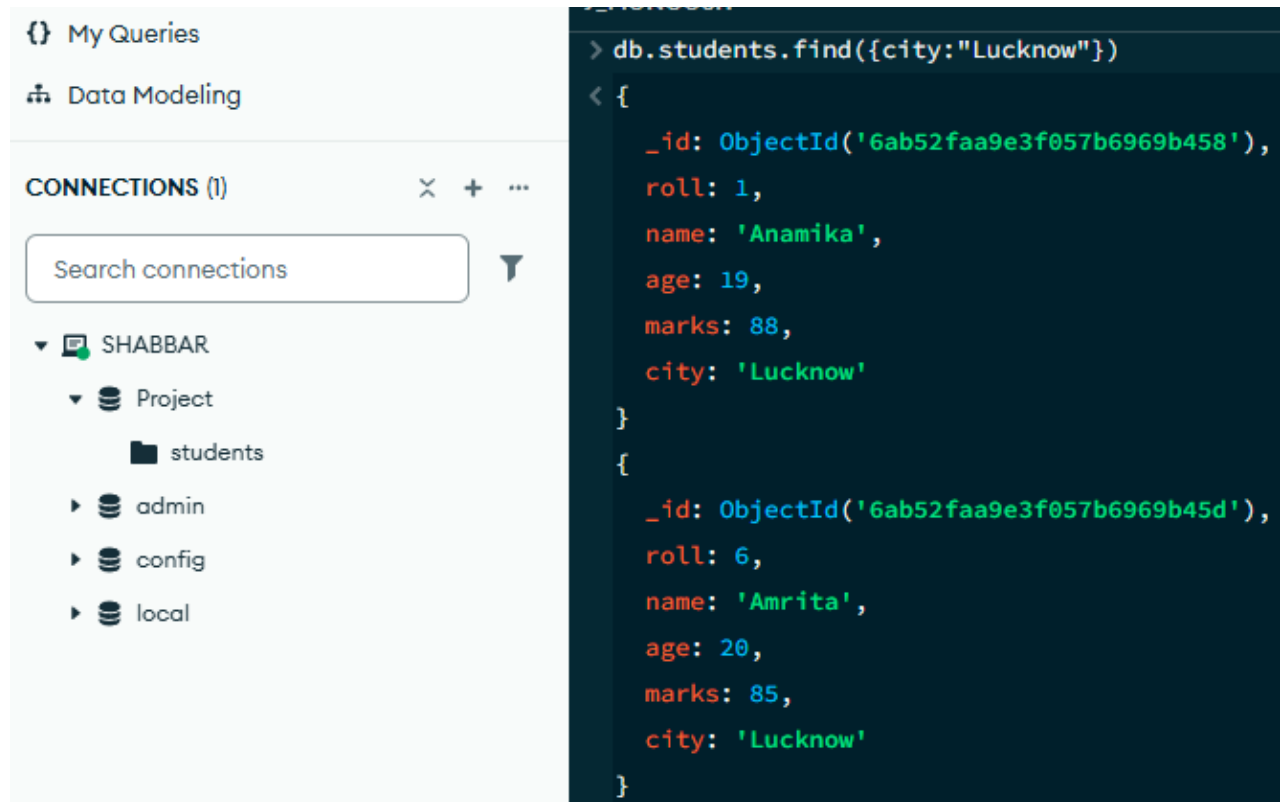
```
{
  _id: ObjectId('6ab52faa9e3f057b6969b45a'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
  marks: 92,
  city: 'Kanpur'
}
{
  _id: ObjectId('6ab52faa9e3f057b6969b45b'),
  roll: 4,
  name: 'Annu',
  age: 18,
  marks: 69,
  city: 'Ayodhya'
}
{
  _id: ObjectId('6ab52faa9e3f057b6969b45c'),
  roll: 5,
  name: 'Vansh',
  age: 19,
  marks: 95,
  city: 'Varanasi'
}
```

17. SEARCH ON THE BASIS OF A DATA

Explanation: Finds all students who have the given data.

MongoDB Query: *db.students.find({city:"Lucknow"})*

Here, all those students who live in Lucknow city will be shown.



18. COUNT OPERATION

Explanation: Counts documents currently present in the collection.

MongoDB Query:

```
> db.students.countDocuments ()
< 46
```

NOTE:-Here, the count appears to be 46 because 4 documents have been deleted previously in the DELETE OPERATION.

MONGODB OPERATORS USED:

Operator	Meaning	Example
\$eq	Equal to	marks: {\$eq:88}
\$gt	Greater than	marks: {\$gt:85}
\$lt	Less than	marks: {\$lt:60}
\$gte	Greater than or equal to	marks: {\$gte:90}
\$lte	Less than or equal to	marks: {\$lte:70}
\$and	All conditions true	\$and:[...]
\$or	At least one condition true	\$or:[...]

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update, and delete documents. I also learned how to Ascending order & descending order data, comparison operators such as \$eq, \$gt, \$lt, \$gte, and \$lte work, along with logical operators such as \$and and \$or.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.